

System Design

23CSH-314

Assignment



Submitted To:

Er. Alok Kumar

E17196

Submitted by:

Johnson Kumar

23BCS12654

KRG- 1A

Q1. Explain SRP and OCP in detail with proper examples**Solution:**Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that a class should have only one responsibility and therefore only one reason to change. This means that a class should focus on a single task or concern in the system. If a class performs multiple unrelated functions, changes in one function may affect others, making the system complex, error-prone, and difficult to maintain.

SRP promotes high cohesion and low coupling, leading to better software design.

=>Example Violating SRP

```
class LibraryManager {  
    void issueBook() {  
    }  
    void saveToDatabase() {  
    }  
    void sendNotification() {  
    }  
}
```

This class handles:

- Library business logic
- Database operations
- Notification services

Any change in storage or messaging requires modifying the same class, violating SRP.

=>Correct Example (SRP Applied)

```
class LibraryService {  
    void issueBook() {  
    }  
}  
class LibraryRepository {  
    void save() {  
    }  
}  
class NotificationService {  
    void notifyUser() {  
    }  
}
```

Each class now has one well-defined responsibility, improving maintainability and testability.

Open/Closed Principle (OCP)

The Open/Closed Principle (OCP) states that software entities should be open for extension but closed for modification. This means new functionality should be added by extending existing code rather than changing it, which reduces the risk of introducing bugs into already tested code.

This principle is commonly implemented using interfaces, abstract classes, and polymorphism.

=>Example Violating OCP

```
class ShippingCostCalculator {
    double calculate(String mode, double weight) {
        if (mode.equals("Air"))
            return weight * 10;
        else if (mode.equals("Road"))
            return weight * 5;
        else if (mode.equals("Sea"))
            return weight * 3;
        return 0;
    }
}
```

Whenever a new shipping method is introduced, this class must be modified, which violates OCP and increases the risk of defects.

=>Correct Example (OCP Applied)

```
interface ShippingMode {
    double calculateCost(double weight);
}

class AirShipping implements ShippingMode {
    public double calculateCost(double weight) {
        return weight * 10;
    }
}

class RoadShipping implements ShippingMode {
    public double calculateCost(double weight) {
        return weight * 5;
    }
}

class SeaShipping implements ShippingMode {
    public double calculateCost(double weight) {
        return weight * 3;
    }
}
```

Now new shipping methods can be added without modifying existing code, satisfying OCP.

```
class ShippingService {
    double getCost(ShippingMode mode, double weight) {
        return mode.calculateCost(weight);
    }
}
```

Q2. Discuss in detail the violations of SRP and OCP along with their fixes

Solution:

Violations of SRP

=> Violation Example

```
class BankAccount {
    void deposit() {}
    void withdraw() {}
    void generateStatement() {}
    void saveToDatabase() {}
}
```

This class performs:

- Core banking operations
- Report generation
- Data persistence

These responsibilities belong to different layers of the application. Combining them causes:

- Difficulty in maintenance
- Increased coupling
- Higher chance of errors
- Harder unit testing

Any change in database technology or reporting format would force changes in this class, even though the banking logic remains the same — thus violating SRP.

=> Fix (SRP Applied)

```
class BankAccount {
    void deposit() {}
    void withdraw() {}
}
class AccountRepository {
    void save(BankAccount acc) {}
}
class StatementGenerator {
    void generate(BankAccount acc) {}
}
```

Now each class has one clearly defined responsibility:

- BankAccount handles business logic
- AccountRepository handles persistence
- StatementGenerator handles reporting

This improves modularity, maintainability, testability, and ensures that changes in one concern do not affect others.

Violations of OCP

=> Violation Example

```
class TaxCalculator {
    double calculate(String country, double income) {
        if (country.equals("India"))
            return income * 0.18;
        else if (country.equals("USA"))
            return income * 0.25;
        else if (country.equals("UK"))
            return income * 0.20;
        return 0;
    }
}
```

This design requires modifying the TaxCalculator class whenever tax rules for a new country are added.

- Breaks the Open/Closed Principle
- Risks introducing bugs in existing logic
- Makes the system harder to extend
- Violates clean architecture practices

=> Fix (OCP Applied)

```
interface TaxStrategy {
    double calculateTax(double income);
}

class IndiaTax implements TaxStrategy {
    public double calculateTax(double income) {
        return income * 0.18;
    }
}

class USATax implements TaxStrategy {
    public double calculateTax(double income) {
        return income * 0.25;
    }
}

class UKTax implements TaxStrategy {
    public double calculateTax(double income) {
        return income * 0.20;
    }
}

class TaxService {
    double calculate(TaxStrategy tax, double income) {
        return tax.calculateTax(income);
    }
}
```

Now new tax rules can be added by creating new classes that implement TaxStrategy, without modifying existing tested code. This ensures:

- Better extensibility
- Reduced risk of defects
- Strong compliance with OCP

Conclusion

- SRP ensures that each class has only one responsibility, improving maintainability and clarity.
- OCP ensures that software components can be extended without being modified, improving scalability and reliability.
- Violations of both principles result in rigid, fragile, and hard-to-maintain systems, while their fixes promote clean, modular, and robust design.